lezione del 31-03-2026

la lezione del 30-03-2026 l'ho saltata
guardala online

Lezione 13
Corso 1

Lifetime

Riferimenti ed esistenza in vita

© G. Malnati, 2021-24

Riferimenti e funzioni

- Se una funzione riceve un parametro di tipo riferimento (mutabile o meno), il tempo di vita del riferimento diventa parte integrante della firma della funzione
  - E' necessario infatti che tutte le operazioni eseguite dalla funzione sul riferimento **siano compatibili** con la validità del dato memorizzato al suo interno
  - `fn f(p: &i32) { … }` ⇒ `fn f<'a>(p: &'a i32) { … }`

- Il compilatore provvede in molti casi a effettuare autonomamente la riscrittura indicata senza il bisogno dell'intervento del programmatore (*lifetime elision*)

bisogna fornire informazioni al compilatore per fargli fare l'analisi statica specificando i legami tra riferimenti in ingresso e riferimenti in uscita alla funzione

in molti casi il compilatore non ha bisogno di avere dettagli ulteriori forniti dal programmatore —> lifetime elision

l'annotazione del tempo di vita serve per propagare il tempo di vita; se non la uso il compilatore esegue il lifetime elision.

di default l'annotazione del tempo di vita viene assegnata dal compilatore. QUi il tempo di vita del valore di ritorno dipende dal tempo di vita del primo parametro, quindi stiamo specificando un legame tra il riferimento in ingresso e quello d'uscita

Riferimenti e funzioni

- Uno dei casi più frequenti in cui il compilatore non riesce a dedurre il corretto tempo di vita è legato a funzioni che **restituiscono un riferimento**, estratto da una delle strutture dati ricevute in ingresso
- Se una funzione ha due o più riferimenti in ingresso e un riferimento in uscita, il compilatore non è in grado di decidere (senza eseguire il codice) da quale parametro provenga il riferimento in uscita

  queste sono annotazioni del tempo di vita: c'è un apice seguito da un simbolo, tendenzialmente una lettera minuscola
  - L'inserimento degli identificatori nella firma della funzione risolve questa ambiguità
  - Occorre **annotare** il tipo restituito con la corretta **etichetta**
- Se la funzione riceve **più riferimenti**, può essere necessario indicare se il loro tempo di vita sia vincolato al più breve o se siano disgiunti
  - Nel primo caso si usa un solo identificatore `fn f<'a>(p1: &'a i32, p2:&'a i32) -> & 'a i32 { … }`
  - Nel secondo si usano etichette diverse `fn f<'a, 'b>(p1: &'a i32, p2:&'b i32) -> & 'a i32 { … }`
- Le **annotazioni** del tempo di vita equivalgono ad **etichette** che sono applicate ai riferimenti per dire al compilatore che un riferimento vivrà almeno quanto quest'altro riferimento. Non cambiano la durata effettiva dei dati a cui i riferimenti puntano, ma aiutano il compilatore a verificare che tutti i riferimenti siano sempre validi e non diventino "pendenti" (dangling) puntando a memoria deallocata.

Tuttavia non è sempre necessario esplicitare le annotazioni: se non ci sono ambiguità per il compilatore allora non bisogna specificare, altrimenti si

Regole del compilatore

- Rust, per default, provvede ad assegnare, ad ogni riferimento presente tra i parametri di una funzione (o in una struttura dati) un tempo di vita distinto:
  - Una funzione con un parametro ha un parametro di lifetime ('a)
  - Una funzione con due parametri ha due parametri di lifetime ('a e ‘b)

- Tali tempi di vita si propagano al codice che fa uso di tale struttura dati e, in assenza di ambiguità, non richiede di esplicitare gli identificatori dei tempi di vita

Text

Elisione dei tempi di vita

1. Se c'è un solo parametro di lifetime in input, quel parametro è assegnato a tutti i parametri di lifetime di output
2. Non ci sono riferimenti in output
3. Se ci sono più parametri di lifetime di ingresso, ma uno di loro è &self o &mut self (poiché si tratta di un metodo) il parametro di lifetime di self è assegnato a tutti i parametri di lifetime di output
   - Questo parte dal presupposto che se un metodo di un oggetto restituisce un dato preso in prestito, questo sia stato preso dai dati posseduti dall'oggetto stesso

11:47

```
fn get_first_element(arr: &[i32]) -> &i32 {
    if arr.is_empty() {
        panic!("Array is empty");
    }
    &arr[0]
}

fn main() {
    let array = [1, 2, 3, 4];
    println!("First element: {}", get_first_element(&array));
}
```

qui non è necessario specificare dei dettagli relativamente alle annotazioni del tempo di vita, perchè il compilaore può dedurlo

```rust
fn trova_primo(s: &str, target: char) -> Option<&str> {
    for (i, c) in s.chars().enumerate() {
        if c == target {
            return Some(&s[..i]);
        }
    }
    None
}
fn main() {
    let s = String::from("hello");
    let result;
    {
        let r = trova_primo(&s, 'l');
        result = r.unwrap();
    }
    println!("{}", result); // Stampa "he
}
```

11:50

```rust
struct Example {
    data: i32,
}

impl Example {
    fn get_data_ref(&self) -> &i32 {
        &self.data
    }
}

fn main() {
    let ex = Example { data: 42 };
    let data_ref = ex.get_data_ref();
    println!("Data reference: {}", data_ref);
}
```

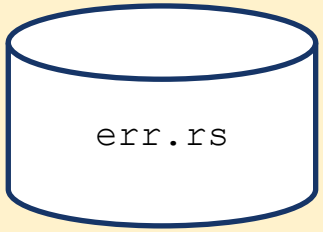
qui il legame è implicito, c'è un riferimento al parametro in ingresso e in output si restituisce un riferimento a self. QUindi il compilatore lo capisce.

```rust
struct Example {
    data1: i32,
    data2: i32,
}
impl Example {
fn get_data_ref(&self, other: &i32) -> &i32 {
        if self.data1 > *other {
            &self.data1
        } else {
            &self.data2
        }
    }
}
fn main() {
    let ex = Example { data1: 42, data2: 20 };
    let other_data = 30;
    let data_ref = ex.get_data_ref(&other_data);
    println!("Data reference: {}", data_ref);
}
```

il compilatore applica la seguente regola: se c'è un &self, si assume che il riferimento al valore di ritorno sia legato al parametro &self. Pertanto non occorre annotare nulla

```rust
struct Example {
    data1: i32,
    data2: i32,
}
impl Example {
    fn get_data_ref(&self, other: &i32) -> &i32 {
        if self.data1 > *other {
            &self.data1
        } else {
            &other
        }
    }
}
fn
```

11:53



qui il valore di ritorno può dipendere sia da self, sia da other, per cui la regola scritta nella slide prima non vale. Quindi questo richide specificare l'annotazione del tempo di vita

```
error: lifetime may not live long enough
  --> src\main.rs:10:13
   |
6  |     fn get_data_ref(&self, other: &i32) -> &i32 {
   |                     -             - let's call the lifetime of this
reference `'1`
   |                     |
   |                     let's call the lifetime of this reference `'2`
...
10 |             &other
   |             ^^^^^^ method was supposed to return data with lifetime `'2`
but it is returning data with lifetime `'1`
}
```

```rust
struct Example {
    data1: i32,
    data2: i32,
}
impl Example {
fn get_data_ref<'a>(&'a self, other: &'a i32) -> &'a i32 {
        if self.data1 > *other {
            &self.data1
        } else {
            &other
        }
    }
}
fn main() {
    let ex = Example { data1: 42, data2: 20 };
    let other_data = 30;
    let data_ref = ex.get_data_ref(&other_data);
    println!("Data reference: {}", data_ref);
}
```

11:55

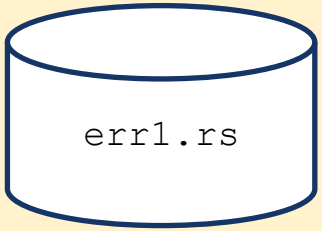
si usa la stessa lettera perchè qui l'uscita può dipendere da entrambi, per cui il compilatore deve tenere conto sia del tempo di vita del primo che del tempo di vita del secondo parametro

elision5.rs

```rust
fn confronta(str1: &str, str2: &str) -> &str {
    if str1.len() > str2.len() { str1 } else { str2 }
}
fn main() {
    let s1 = String::from("hello");
    let s2 = String::from("world");
    let risultato;
    risultato = confronta(&s1, &s2);

    println!("La stringa più lunga è: {}", risultato);
}
```

poichè ci sono più parametri in ingresso, il compilatore si chiede sempre da che cosa dipende l'uscita. DIpende dal primo, dal secondo o da entrambi?



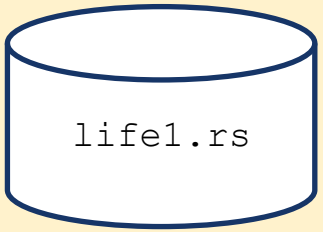
il compilatore chiede di SPECIFICARE L'ANNOTAZIONE DEL TEMPO DI VITA

```
1 | fn confronta(str1: &str, str2: &str) -> &str {
  |                    ----        ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the
signature does not say whether it is borrowed from `str1` or `str2`
help: consider introducing a named lifetime parameter
  |
1 | fn confronta<'a>(str1: &'a str, str2: &'a str) -> &'a str {
  |             ++++        ++             ++          ++
```

- Le **annotazioni** devono essere riportate solo nella signature della funzione e non nel corpo della funzione.

```
fn confronta<'a> (str1: &'a str, str2: &'a str) -> &'a str {
    if str1.len() > str2.len() {
        str1
    } else {
        str2
    }
}
fn main() {
    let s1 = String::from("hello");
    let s2 = String::from("world!");
    let risultato;
    risultato = confronta(&s1, &s2);

    println!("La stringa più lunga è: {}", risultato);
}
```



- Se il riferimento restituito dipende solo da un parametro, occorre specificare l'annotazione solo per il parametro per cui c'è la dipendenza. 12:01

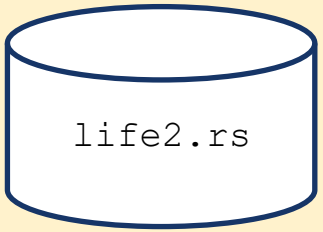
```
fn stampa<'a> (str1: &'a str, str2: &str) -> &'a str {
    println!("{}",str2);
    str1

}

fn main() {
    let s1 = String::from("Viva i lifetimes");
    let s2 = String::from("Questa è una stringa di benvenuto");

    let risultato;
    risultato = stampa(&s1, &s2);

    println!("{}", risultato);
}
```



- Annotazione equivalente alla precedente

```
fn stampa<'a, 'b> (str1: &'a str, str2: &'b str) -> &'a str {
    println!("{}",str2);
    str1

}

fn main() {
    let s1 = String::from("Viva i lifetimes");
    let s2 = String::from("Questa è una stringa di benvenuto");

    let risultato;
    risultato = stampa(&s1, &s2);

    println!("{}", risultato);
}
```

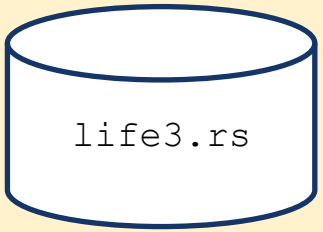
- Nel caso in cui la funzione sia generica, le meta-variabili di tipo vengono indicate dopo gli identificatori del tempo di vita.

```
fn merge_slices<'a, T>(first: &'a [T], second: &'a [T]) -> Vec<&'a T>
    where T: Copy {
    let mut merged: Vec<&'a T> = Vec::new();
    for item in first.iter().chain(second.iter()) {
        merged.push(item);
    }
    merged
}

fn main() {
    let first_slice = &[1, 2, 3, 4];
    let second_slice = &[5, 6, 7, 8];

    let merged_result = merge_slices(first_slice, second_slice);

    println!("Merged slices: {:?}", merged_result);
}
```



```rust
struct S(u8);

fn f<'a>(x: &S, y: &'a S) -> &'a u8 {
    &y.0
}

fn print_byte(byte: &u8) {
    println!("Byte: {}", byte);
}

fn main() {
  let v1 = S(1);
  let mut v2 = S(2);

  let r = f(&v1, &v2);

  v2 = v1;

  print_byte(r);
}
```

31-03-2026 12:06

err2.rs

anche se viene restituito un campo del riferimento alla struct, io sto comunque creando un altro riferimento alla struct originale. Per cui non posso modificare il valore di v2 tramite movimento e poi stampare un riferimento, perchè viola la regola del borrow checker

```rust
struct S(u8);

fn f<'a>(x: &S, y: &'a S) -> &'a u8 {
    &y.0
}

fn print_byte(byte: &u8) {
    println!("Byte: {}", byte);
}

fn main() {
  let v1 = S(1);
  let mut v2 = S(2);

  let r = f(&v1, &v2);

  v2 = v1;

  print_byte(r);
}
```

```
15 |     let r = f(&v1, &v2);
   |                    --- `v2` is borrowed here
16 |
17 |     v2 = v1;
   |     ^^^^^^^ `v2` is assigned to here but it was
already borrowed
18 |
19 |     print_byte(r);
   |                - borrow later used here
```

```rust
struct S(u8);

fn f<'a>(x: &S, y: &'a S) -> &'a u8 {
    &y.0
}

fn print_byte(byte: &u8) {
    println!("Byte: {}", byte);
}

fn main() {
    let v1 = S(1);
    let mut v2 = S(2);

    let r = f(&v1, &v2);
    print_byte(r);
    v2 = v1;
}
```

12:14
qui c'è il concetto di shadowing, per cui sto riassegnando la variabile v2 tramite let.

```rust
struct S(u8);

fn f<'a>(x: &S, y: &'a S) -> &'a u8 {
    &y.0
}

fn print_byte(byte: &u8) {
    println!("Byte: {}", byte);
}

fn main() {
    let v1 = S(1);
    let v2 = S(2);

    let r = f(&v1, &v2);

    let v2 = v1;

    print_byte(r);
}
```

'static

12:17

con l'attributo 'static definisco il tempo di vita della variabile uguale al tempo di vita totale del programma.

- Se la funzione restituisce un riferimento ad una variabile locale che contiene una stringa costante, bisogna riportare l'annotazione di lifetime `'static` che indica che il riferimento restituito ha una durata statica, cioè per l'intera durata del programma.

```
fn crea_stringa() -> &'static str {
    let s = "hello";
    s
}

fn main() {
    let s = crea_stringa();
    println!("{}", s);
}
```

Riferimenti e funzioni

- Lo scopo degli identificatori relativi al ciclo di vita è **duplice**:
  - Per il codice che **invoca** la funzione (chiamante), essi indicano su **quale**, tra gli indirizzi in ingresso, è basato il risultato in uscita
  - Per il codice **all'interno** della funzione (chiamata), essi garantiscono che vengano restituiti solo indirizzi cui **è lecito accedere** per (almeno) il tempo di vita indicato
- All'atto dell'invocazione, gli identificatori forniti dal programmatore (o inseriti automaticamente dal compilatore, quando possibile) sono legati all'effettivo intervallo minimo (espresso come insieme di linee di codice) nel quale il valore da cui il prestito è stato preso debba restare bloccato per non violare le assunzioni su cui la funzione è basata
  - Tentativi di modificare il valore originale da cui il prestito è preso prima che il tempo di vita sia trascorso portano ad errori di compilazione che costituiscono la base della robustezza del sistema di possesso e prestito offerto da Rust

© G. Malnati, 2021-24

se ad una funzione passiamo una variabile, essa potrebbe contenere al suo interno dei riferimenti. Quindi dobbiamo essere sicuri che i tempi di vita memorizzati nella struttura dati durino almeno quanto la durata della struttura dati stessa

Riferimenti, funzioni e strutture dati

- Se la funzione riceve come parametri un riferimento ad una struttura dati contenente un riferimento ed un altro riferimento, il compilatore non può inferire la relazione tra **il tempo di vita della struttura** in cui il riferimento è memorizzato e **il riferimento stesso**.
- Occorre specificare al compilatore se esiste una relazione tra il riferimento passato come parametro e quello contenuto nella struttura dati (caso più frequente) oppure non esiste alcuna relazione.

© G. Malnati, 2021-24

```rust
fn insert (vet: &mut Vec<&str>, s: &str)  {
    vet.push(s);
}
fn main() {
    let mut v = Vec::<&str>::new();
    {
        let s ="Ciao".to_string();
        insert(&mut v, &s);
    }
    println!("{:?}", v);
}
```

se il parametro contiene un riferimento, la signature qui non va bene. Le annotazioni del tempo di vita servono al compilatore perchè

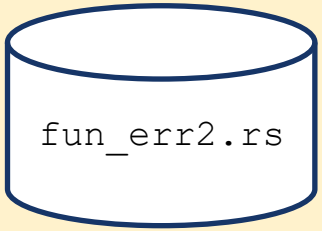
tuttavia il contenitore ha un contenuto che non dura quanto lui. QUindi i riferimenti contenuti nella struttura dati devono durare almeno quanto la durata del contenitore stesso; questa è una regola del borrow checker.

```
error: lifetime may not live long enough
 --> src\main.rs:2:1
  |
1 | fn insert (vet: &mut Vec<&str>, s: &str)  {
  |                         -         - let's call the lifetime of this
reference `’1`
                          let's call the lifetime of this reference `'2`
2 | vet.push(s);
  | ^^^^^^^^^^^ argument requires that `'1` must outlive `'2`
help: consider introducing a named lifetime parameter
1 | fn insert<'a> (vet: &mut Vec<&'a str>, s: &'a str)  {
```

qui il dato di s, puntato da &s, muore dentro lo scope interno. Poichè il metodo inserisce un riferimento ad un dato che non esiste più il tempo di vita del dato è finito prima del contenitore in cui è contenuto

```rust
fn insert <'a> (vet: &mut Vec<&'a str>, s: &'a str)  {
    vet.push(s);
}
fn main() {
    fn main() {
        let mut v = Vec::<&str>::new();
        {
            let s ="Ciao".to_string();
            insert(&mut v, &s);
}
```

Text



```
error[E0597]: `s` does not live long enough
  --> src\main.rs:9:28
   |
 6 |         let mut v = Vec::<&str>::new();
   |             ----- variable `v` declared here
 7 |         {
 8 |             let s ="Ciao".to_string();
   |                 - binding `s` declared here
 9 |             insert(&mut v, &s);
   |                            ^^ borrowed value does not live long enough
10 |         }
   |         - `s` dropped here while still borrowed
11 |         println!("{:?}", v);
   |                          - borrow later used here
```

Riferimenti, funzioni e strutture dati

```
fn insert <'a> (vet: &mut Vec<&'a str>, s: &'a str)  {
    vet.push(s);
}
fn main() {
        let mut v = Vec::<&str>::new();
        let s ="Ciao".to_string();
        insert(&mut v, &s);
        println!("{:?}", v);
}
```

questa è la versione corretta del programma precedente.

Riferimenti, funzioni e strutture dati

```
fn insert<'a, 'b>(vet: &mut Vec<&'a str>, s: &'b str)  {
    vet.push("1");
    println!("{}", s);
}

fn main() {
    let mut v = Vec::<&str>::new();
    insert(&mut v, & "Inserisco una stringa".to_string());
    println!("{:?}", v);
}
```

- Non esiste alcuna relazione tra la stringa s e la stringa contenuta dentro il vettore.

```rust
fn insert<'a>(vet: &mut Vec<&'a str>, s: &'a str)  {
    vet.push(s);
}

fn main() {

    let mut v = Vec::<&str>::new();

    insert(&mut v, & "Ciao".to_string());

    println!("{:?}", v);
}
```

la variabile vive solo nello scope della funzione pertanto non posso stampare il vettore

Perché questo programma non è compilato?

Rispondete sul canale Discord Rust

https://discord.gg/Y7kjx8T2

```rust
fn insert(vet: &mut Vec<String>, s: &str)  {
    vet.push(s.to_string());
}
fn main() {
    let mut v = Vec::<String>::new();
    {
        let messaggio = String::from("Ciao");
        insert(&mut v, &messaggio);
        println!("{:?}", v);
    }
    println!("{:?}", v); // funziona!
}
```

il vettore qui contiene il dato, pertanto è il possessore. La stringa viene mossa dentro il vec, per cui il vettore si porta dietro il dato stesso

challenge2.rs

quindi anche se il messagio muore dentro questo blocco, il vettore può essere correttamente stampato

Perché questo programma viene compilato correttamente senza specificare annotazioni di lifetime?

Rispondete sul canale Discord Rust

https://discord.gg/Y7kjx8T2

12:39

per ogni struttura che contiene un riferimento, il riferimento deve essere sempre annotato. Questa regole deve essere sempre applicata.

Riferimenti e tipi composti (struct ed enum)

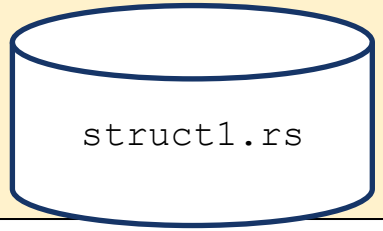
- Lo stesso tipo di ragionamento vale se il dato viene salvato all'interno di una qualsiasi struttura dati
  - Rust verifica che il valore a cui si punta abbia un tempo di vita maggiore o uguale del tempo di vita della struttura dati
  - Questo richiede di esplicitare il tempo di vita della struttura rispetto al tempo di vita dei riferimenti in essa contenuti
- Un riferimento che cessa di vivere si propaga a tutti i contenitori

```
struct User<'a> {
  id: u32,
  name: &'a str,
}
```

```rust
struct Contenitore<'a> {
    dati: &'a str,
}

fn main() {
    let dati = String::from("Dati importanti");
    let cont = Contenitore { dati: &dati };
    println!("{}", cont.dati);
}
```

Il lifetime deve essere specificato per garantire che il riferimento del campo `dati` sia valido e non punti a una memoria non allocata.

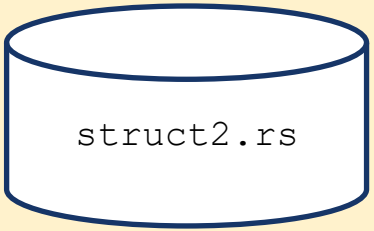


12:41

```rust
struct TextWindow<'a> {
    content: &'a str,
}
impl<'a> TextWindow<'a> {
    fn new(content: &'a str) -> Self {
        TextWindow { content }
    }
    fn display(&self) {
        println!("Text window content: {}", self.content);
    }
}
fn main() {
    let my_text = "Hello, world!".to_string();
    let text_window = TextWindow::new(&my_text);
    text_window.display();
}
```

Quando si implementano metodi per struct con annotazioni di lifetime, bisogna includere le stesse annotazioni nel blocco impl.

qui serve specificare l'annotazione del tempo di vita perchè manca &self. Devo quindi specificare il legame tra la stringa e il valore di ritorno



Riferimenti e strutture dati

- Se una struttura che contiene riferimenti è contenuta, a sua volta, in un'altra struttura, anche quest'ultima deve avere il tempo di vita specificato in modo esplicito

```
struct User<'a> {
  id: u32,
  name: &'a str,
}
```

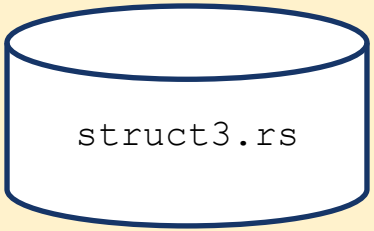
```
struct Data<'a> {
  user: User<'a>,
  password: String,
}
```

```rust
#[derive(Debug)]
struct User<'a> {
    id: u32,
    name: &'a  str,
}

struct Data<'a> {
    user: User<'a>,
    password: String,
}

fn main() {
    let user = User { id: 1, name: "Alice" };
    let data = Data { user, password: String::from("password123") };

    println!("Data: {:?}, {:?}", data.user, data.password);
}
```

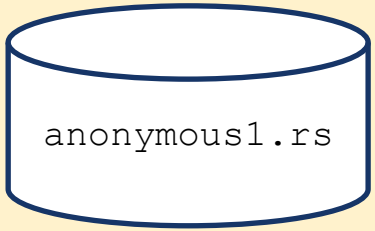


Anonymous Lifetime

12:44 30-03-2026

- Nel caso di riferimenti inseriti in strutture dati, se la definizione del metodo permette di ricavare in modo implicito la dipendenza dei tempi di vita, si può utilizzare l'annotazione di *anonymous lifetime (`<'_>`).*

```
struct Worker<'a> {
    name: &'a str,
    id: u32,
}
impl Worker<'_> {
    fn new(name: &str, id: u32) -> Worker {
        Worker { name, id }
    }
    fn get_name(&self) -> &str {
        self.name
    }
}
fn main() {
    let name = "Alice";
    let id = 1001;
    let worker = Worker::new(name, id);
    println!("Worker name: {}", worker.get_name());
}
```



Trait Bound, Generic Type e Lifetimes

```
use std::fmt::Display;
fn longest_with_an_announcement<'a, T>(x: &'a str, y: &'a str, ann: T) -> &'a str
    where
        T: Display,
{
    println!("Announcement! {}", ann);
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
fn main() {
    let x = "hello";
    let y = "world!";
    let ann = "Important message";
    let longest_str = longest_with_an_announcement(x, y, ann);
    println!("The longest string is: {}", longest_str);
}
```

Per saperne di più

- Diving Deep: How Rust Lifetimes Work
  - https://medium.com/@luishrsoares/diving-deep-how-rust-lifetimes-work-2692633ab344
  - Articolo di approfondimento sui lifetimes

Glossario

- **Annotazione** del tempo di vita: etichette applicate ai riferimenti per definire le relazioni tra i tempi di vita dei riferimenti
- **Lifetime elision**: il compilatore applica un insieme di regole per inferire automaticamente i tempi di vita
- **Anonymous Lifetime**: nel caso di implementazioni di metodi per tipi composti si può utilizzare un'annotazione semplificata se la dipendenza dei riferimenti è implicita e può essere inferita dal compilatore.

© G. Malnati, 2021-24